Part 1

HW5
November 27, 2025 8:44 PM

Problem 1:

1. $X = (x, y, z, w)^T$

$\pi_\infty^T X = 0$ with $\pi_\infty = (0,0,0,1)^T$

2. $H = \begin{bmatrix} A & t \\ 0^T & 1 \end{bmatrix}$

Infinity $X = (x, 0)^T$

$X' = HX = \begin{bmatrix} A & t \\ 0^T & 1 \end{bmatrix} \begin{bmatrix} V \\ 0 \end{bmatrix} = \begin{bmatrix} AV \\ 0 \end{bmatrix}$

0 is still on plane $w = 0$

$\therefore$ fixed plane under any affine transformation.

3. $\pi^T X = 0$, $\pi = (a, b, c, d)^T$

$X' = HX \Rightarrow X = H^{-1} X'$

$0 = \pi^T X = \pi^T H^{-1} X' = (H^{-T} X)^T X'$

$\therefore \pi' = H^{-T} \pi$

Problem 3:

1. $x \sim K[I|0]X$, $\hat{X} = K^{-1}x$

$\hat{X}^T \hat{X} = 1$

$(K^{-1}x)^T (K^{-1}x) = x^T K^{-T} K^{-1} x = 1$

$x^T \omega x = 1$, $\omega = K^{-T} K^{-1}$

$\therefore \omega = (KK^T)^{-1}$

2. $\hat{x}_i = K^{-1} x_i$, $i = 1, 2$

$\hat{x}_1^T \hat{x}_2 = 0$

$0 = (K^{-1} x_1)^T (K^{-1} x_2)$

$= x_1^T K^{-T} K^{-1} x_2$

$= x_1^T \omega x_2$

$\therefore x_1^T \omega x_2 = 0$

Problem 2:

1. $X = (x, y, 0, 1)^T$

$x \sim PX = K[r_1 r_2 r_3 t] \begin{bmatrix} x \\ y \\ 0 \\ 1 \end{bmatrix}$

$= K(x r_1 + y r_2 + t)$

$= K[r_1 r_2 t] \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$

2. $L^T X = 0$

$0 = L^T X \sim L^T P X = (P^T L)^T X$

Satisfys $\Pi^T X = 0$, $\Pi = P^T L$

$\therefore$ Back projection of image line is 3D plane Π

3. $P = K[R|t]$, $P' = K'[R|t]$

$x \sim K[R|t]X$, $x' \sim K'[R|t]X$

$\tilde{x} = K^{-1} x \sim [R|t]X$, $\tilde{x}' = K'^{-1} x' \sim [R|t]X$

$\therefore K'^{-1} x' \sim K^{-1} x \Rightarrow x' \sim K' K^{-1} x$

$\therefore H = K' K^{-1}$

3. $x \sim H X_P$, $H = [h_1 h_2 h_3]$

$e_1 = (1, 0, 0)^T$, $e_2 = (0, 1, 0)^T$

$x_1 \sim H e_1 = h_1$, $x_2 \sim H e_2 = h_2$

$x_1^T \omega x_2 = 0 \Rightarrow h_1^T \omega h_2 = 0$

$x_1^T \omega x_1 = x_2^T \omega x_2 \Rightarrow h_1^T \omega h_1 = h_2^T \omega h_2$

$\therefore$ From $H = [h_1 h_2 h_3]$, we get

$\begin{cases} h_1^T \omega h_2 = 0 \\ h_1^T \omega h_1 = h_2^T \omega h_2 \end{cases}$

4. $K = \begin{bmatrix} f & 0 & V_0 \\ 0 & f & V_0 \\ 0 & 0 & 1 \end{bmatrix}$

$KK^T = \begin{bmatrix} f^2+V_0^2 & U_0V_0 & U_0 \\ U_0V_0 & f^2+V_0^2 & V_0 \\ U_0 & V_0 & 1 \end{bmatrix}$

Skew parameter = 0

Focal length = $f_x = f_y = f$

$W = \begin{bmatrix} W_{11} & W_{12} & W_{13} \\ W_{12} & W_{22} & W_{23} \\ W_{13} & W_{23} & W_{33} \end{bmatrix}$

$W_{12} = 0$

$f_x = f_y \Rightarrow W_{11} = W_{22}$

$\therefore W = \begin{bmatrix} \alpha & 0 & \gamma \\ 0 & \alpha & \beta \\ \gamma & \beta & \eta \end{bmatrix}$

$\therefore W_{12} = 0, \; W_{11} = W_{22}$

0 skew Square pixels

Figure 1: Problems 1, 2, and 3 process and computation

Part 2

The paper presents a flexible and effective method for camera calibration in 3D computer vision. Unlike traditional techniques, this approach does not require prior knowledge of the camera's motion or the movement of the calibration pattern. The only requirement is that the camera captures a planar pattern from multiple orientations.

The method begins with a closed-form solution that is further refined using a nonlinear optimization. It also addresses key steps such as approximating a general 3×3 matrix with a valid rotation matrix, estimating the camera's intrinsic parameters, and computing the homography between the model plane and its image.

The proposed technique has been validated through both real-world experiments and computer simulations, demonstrating strong performance. Compared to older methods that often depend on expensive hardware setups, this approach is more practical, cost-effective, and easier to implement. It represents an important step in making 3D computer vision usable outside controlled laboratory environments.

More broadly, 3D reconstruction blends concepts from computational geometry, image processing, and vision science. The goal is to recover three-dimensional structures from two-dimensional observations—such as photographs or video frames—making it one of the most challenging and intriguing areas in modern computer vision. The theory can be broken down into several components:

Image Capture:
The reconstruction process begins by capturing multiple images of the 3D object from different viewpoints. Collecting several perspectives is essential because it provides enough visual information to recover the object's full structure. In some cases, structured light—projected patterns placed onto the object—is used to help record complex or detailed surfaces more accurately.

Feature Detection and Matching:
Each captured image contains distinct visual features, such as edges, corners, or textured points, which are identified using feature-detection algorithms. After these features are extracted, they are matched across different images to establish correspondences. These matches are critical because they allow the system to determine how the photos relate spatially, forming the foundation for accurate 3D reconstruction.

Estimating Camera Parameters:
Accurate reconstruction requires estimating the camera's intrinsic parameters, such as focal length, principal point, and lens distortion. Understanding these values ensures that image measurements correspond correctly to real-world geometry. It is also necessary to determine the camera's pose—its exact position and orientation for each captured image—since this allows the system to understand how the camera moved relative to the 3D scene.

Triangulation:
Once the corresponding features between images are matched and the camera parameters are known, triangulation is used to compute the 3D coordinates of those points. By intersecting the projection rays extending from each camera viewpoint, the system can estimate where each point lies in three-dimensional space.

Point Cloud Generation and Meshing:
The triangulated 3D points form a point cloud, which serves as a rough representation of the object's structure. To transform this into a continuous surface, a mesh is generated over the

point cloud, often using techniques such as Delaunay triangulation. This produces a smoother, more complete model of the object.

Texture Mapping:
To enhance realism, the original image textures are applied to the reconstructed mesh. This step maps the colours and details from the photographs onto the 3D surface, giving the final model a natural and visually accurate appearance.

Optimization and Refinement:
Finally, the model undergoes refinement through iterative optimization techniques. Methods such as bundle adjustment are used to minimize geometric and projection errors, improving the precision of both the 3D points and the estimated camera parameters.

Stereopsis determines depth by analyzing the parallax between images captured from slightly different viewpoints. Structure from Motion (SfM) extends this idea by using motion across multiple images to estimate both the camera trajectory and the 3D structure of the scene. Simultaneous Localization and Mapping (SLAM), widely used in robotics, builds a map of an unknown environment while simultaneously tracking the position of the moving agent within it. Despite its capabilities, 3D reconstruction faces several challenges: surfaces that are reflective, transparent, or lacking texture can be difficult to model; occlusions and shadows make it hard to reconstruct parts of the object that are hidden in some images; and achieving high-resolution results often requires significant computational resources. With applications ranging from virtual reality to medical imaging, 3D reconstruction continues to advance rapidly as improvements in hardware and software push the field forward.

As for my observations and results, they align well with the findings presented in the provided paper and research. The Camera Calibrator and Reconstruction theory worked well with the two images that I have provided, and helped me understand the research more. A minor error was that the program didn't collect many data points, but that could be due to the nature of the images.

```
I1 = imread('C:\Users\ansug\OneDrive\Documents\MATLAB\Image1.png');
I2 = imread('C:\Users\ansug\OneDrive\Documents\MATLAB\Image2.png');
figure
imshowpair(I1, I2, 'montage');
title('Original Images');
% Load precomputed camera intrinsics
load("calibrationSession.mat");
figure
imshowpair(I1, I2, 'montage');
title('Undistorted Images');
% Detect feature points
imagePoints1 = detectMinEigenFeatures(im2gray(I1), MinQuality=0.1);

% Visualize detected points
figure
imshow(I1, InitialMagnification = 50);
title("150 Strongest Corners from the First Image");
hold on
plot(selectStrongest(imagePoints1, 150));
% Create the point tracker
tracker = vision.PointTracker(MaxBidirectionalError=1, NumPyramidLevels=5);

% Initialize the point tracker
imagePoints1 = imagePoints1.Location;
initialize(tracker, imagePoints1, I1);

% Track the points
[imagePoints2, validIdx] = step(tracker, I2);
```

```
[imagePoints2, validIdx] = step(tracker, I2);
matchedPoints1 = imagePoints1(validIdx, :);
matchedPoints2 = imagePoints2(validIdx, :);

% Visualize correspondences
figure
showMatchedFeatures(I1, I2, matchedPoints1, matchedPoints2);
title("Tracked Features");
% Estimate the fundamental matrix
[E, epipolarInliers] = estimateEssentialMatrix(...
    matchedPoints1, matchedPoints2, intrinsics, Confidence = 99.99);

% Find epipolar inliers
inlierPoints1 = matchedPoints1(epipolarInliers, :);
inlierPoints2 = matchedPoints2(epipolarInliers, :);

% Display inlier matches
figure
showMatchedFeatures(I1, I2, inlierPoints1, inlierPoints2);
title("Epipolar Inliers");
relPose = estrelpose(E, intrinsics, inlierPoints1, inlierPoints2);
% Detect dense feature points. Use an ROI to exclude points close to the
% image edges.
border = 30;
roi = [border, border, size(I1, 2)- 2*border, size(I1, 1)- 2*border];
imagePoints1 = detectMinEigenFeatures(im2gray(I1), ROI = roi, ...
    MinQuality = 0.001);
```

```
% Create the point tracker
tracker = vision.PointTracker(MaxBidirectionalError=1, NumPyramidLevels=5);

% Initialize the point tracker
imagePoints1 = imagePoints1.Location;
initialize(tracker, imagePoints1, I1);

% Track the points
[imagePoints2, validIdx] = step(tracker, I2);
matchedPoints1 = imagePoints1(validIdx, :);
matchedPoints2 = imagePoints2(validIdx, :);

% Compute the camera matrices for each position of the camera
% The first camera is at the origin looking along the Z-axis. Thus, its
% transformation is identity.
camMatrix1 = cameraProjection(intrinsics, rigidtform3d);
camMatrix2 = cameraProjection(intrinsics, pose2extr(relPose));

% Compute the 3-D points
points3D = triangulate(matchedPoints1, matchedPoints2, camMatrix1, camMatrix2);

% Get the color of each reconstructed point
numPixels = size(I1, 1) * size(I1, 2);
allColors = reshape(I1, [numPixels, 3]);
colorIdx = sub2ind([size(I1, 1), size(I1, 2)], round(matchedPoints1(:,2)), ...
    round(matchedPoints1(:, 1)));
color = allColors(colorIdx, :);
```

```
% Create the point cloud
ptCloud = pointCloud(points3D, Color=color);

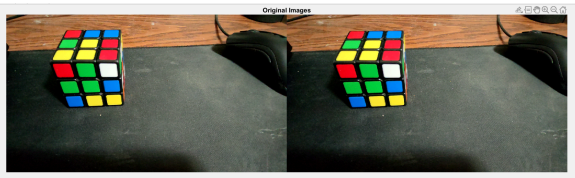
% Visualize the camera locations and orientations
cameraSize = 0.3;
figure
plotCamera(Size=cameraSize, Color="r", Label="1", Opacity=0);
hold on
grid on
plotCamera(AbsolutePose=relPose, Size=cameraSize, ...
    Color="b", Label="2", Opacity=0);

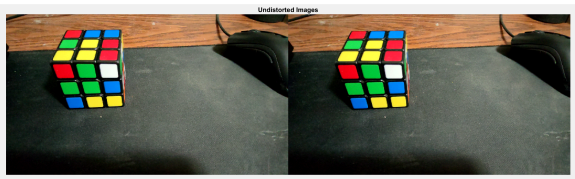
% Visualize the point cloud
pcshow(ptCloud, VerticalAxis="y", VerticalAxisDir="down", MarkerSize=45);

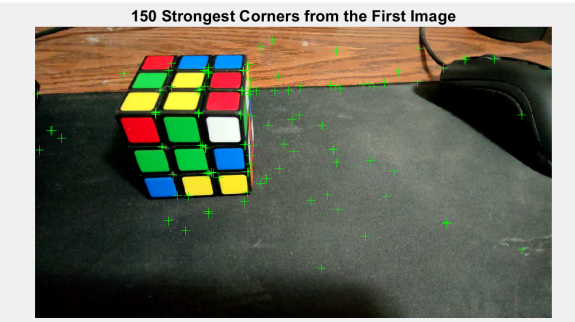
% Rotate and zoom the plot
camorbit(0, -30);
camzoom(1.5);

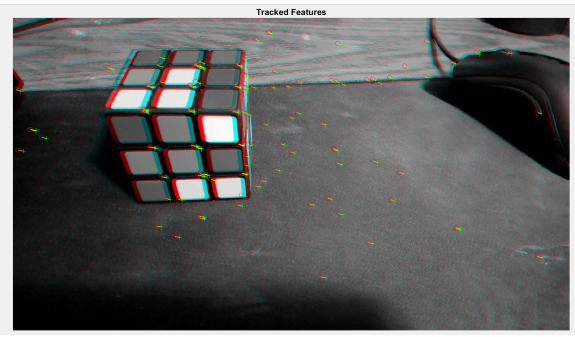
% Label the axes
xlabel("x-axis");
ylabel("y-axis");
zlabel("z-axis")

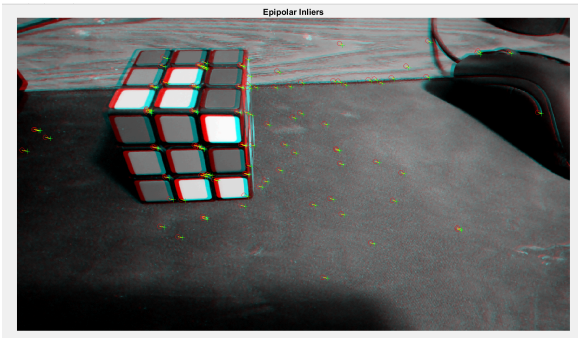
title("Up to Scale Reconstruction of the Scene");
```











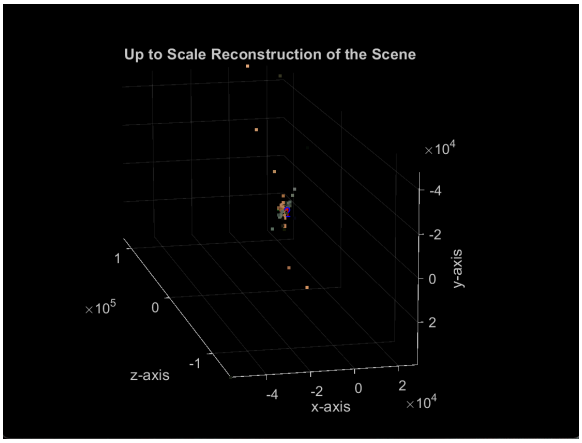


Figure 2: Source Codes and Figures for Part 2